Pipeline Concepts — Complete Solution Set

IBA Karachi · Computer Science · Computer Architecture

Contents

Reference: The Code

```
lw   x1, 0(x10)        # I1
lw   x2, 4(x10)        # I2
add  x3, x1, x2        # I3
mul  x4, x3, x5        # I4
add  x6, x4, x2        # I5
add  x3, x6, x1        # I6
beq  x3, x0, TARGET    # I7
add  x8, x3, x5        # I8
sw   x8, 8(x10)        # I9
TARGET:
sub  x9, x3, x2        # I10
```

Assumptions: Branch at I7 is **NOT taken** at runtime. `x0` = 0, `x5` is nonzero and known at decode time. Pipeline: IF · ID · RN · IS · EX · WB, issue width = 2, execution units: 1 ALU · 1 MUL (3-cycle) · 1 LSU (2-cycle).

1 Q1 — Hazard Identification

All RAW, WAW, and WAR Hazards in I1–I6

Pair	Register	Type	Explanation
I1 → I3	x1	RAW	I3 reads x1 produced by I1
I2 → I3	x2	RAW	I3 reads x2 produced by I2
I3 → I4	x3	RAW	I4 reads x3 produced by I3
I4 → I5	x4	RAW	I5 reads x4 produced by I4
I5 → I6	x6	RAW	I6 reads x6 produced by I5
I1 → I6	x1	RAW	I6 reads x1 — long-distance dependency
I2 → I5	x2	RAW	I5 reads x2 — long-distance dependency
I3 → I6	x3	WAW	Both I3 and I6 write to x3 (output dependency)

Total: 8 hazards (7 RAW + 1 WAW). There are no WAR hazards in I1–I6 at the architectural register level before renaming.

2 Q2 — Register Renaming

Setup

The initial rename table before any instruction executes:

Architectural	Physical
x1	p0
x2	p1
x3	p2
x4	p3
x5	p4
x6	p5

Instruction-by-Instruction Walkthrough

Source registers are looked up in the *current* table before the destination is assigned its new physical register.

I1: `lw x1, 0(x10)` Writes x1 → assign p6. No sources. Renamed: p6 ← `mem[x10+0]`

I2: `lw x2, 4(x10)` Writes x2 → assign p7. No sources. Renamed: p7 ← `mem[x10+4]`

I3: `add x3, x1, x2`

- x1 → p6 (I1), x2 → p7 (I2), writes x3 → p8

Renamed: p8 ← p6 + p7

I4: `mul x4, x3, x5`

- x3 → p8 (I3), x5 → p4 (initial), writes x4 → p9

Renamed: p9 ← p8 × p4

I5: `add x6, x4, x2`

- x4 → p9 (I4), x2 → p7 (I2, never overwritten), writes x6 → p10

Renamed: p10 ← p9 + p7

I6: `add x3, x6, x1`

- x6 → p10 (I5), x1 → p6 (I1, never overwritten), writes x3 → p11

Renamed: p11 ← p10 + p6

Part (a) — Completed Rename Table

Instr	Arch Reg Written	Assigned PRF	Reads (arch → phys)
I1	x1	p6	—
I2	x2	p7	—
I3	x3	p8	x1→p6, x2→p7
I4	x4	p9	x3→p8, x5→p4
I5	x6	p10	x4→p9, x2→p7
I6	x3	p11	x6→p10, x1→p6

Part (b) — Hazards Eliminated vs. Remaining

Eliminated:

- **WAW on x3** — I3 writes p8, I6 writes p11. Completely independent physical registers; both can execute in any order.
- Any **WAR hazards** are dissolved since every destination receives a fresh physical register.

Remaining: All **RAW hazards** persist. Renaming cannot remove true data dependencies — a consumer genuinely needs the value its producer computes, regardless of what physical register it is stored in.

Pair	Physical dep.	Original type	After renaming
I1→I3	p6	RAW	Remains
I2→I3	p7	RAW	Remains
I3→I4	p8	RAW	Remains
I4→I5	p9	RAW	Remains
I5→I6	p10	RAW	Remains
I1→I6	p6	RAW	Remains
I2→I5	p7	RAW	Remains
I3→I6	—	WAW on x3	**Eliminated** ($p8 \neq p11$)

Part (c) — Co-issue Opportunities After Renaming

This part requires separating two distinct constraints:

- **Data dependency constraints** — affected by renaming. Renaming removes WAW and WAR, leaving only RAW.
- **Structural constraints** — imposed by limited hardware (1 ALU, 1 MUL, 1 LSU). Renaming has no effect on these.

Layer 1 — Data dependencies after renaming:

Scanning all 15 pairs, the only pair with no data dependency is **I1 + I2** (independent loads). Every other pair is connected by at least one RAW hazard through the dependency chain.

Layer 2 — Structural constraint on the surviving pair:

I1 and I2 are both loads. With only one LSU port, they cannot issue simultaneously — this is a structural hazard that renaming cannot resolve. The processor serialises them: I1 issues cycle 3, I2 issues cycle 4.

Co-issue pairs with no data dependency after renaming: 1 (I1 + I2).

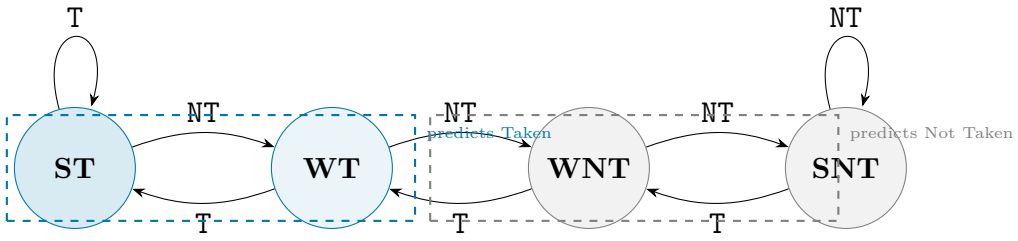
Co-issue pairs valid after applying structural constraints: 0.

The answer depends on which layer the question targets. Both layers must be shown for full marks. Renaming dissolves false dependencies but cannot create hardware resources that do not exist.

3 Q3 — Branch Prediction

The 2-Bit Saturating Counter

State	Code	Prediction
Strongly Taken	ST	Taken
Weakly Taken	WT	Taken
Weakly Not Taken	WNT	Not Taken
Strongly Not Taken	SNT	Not Taken



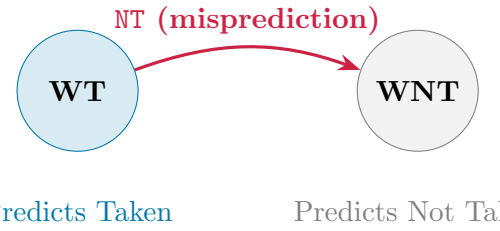
Starting state: **Weakly Taken (WT)**.

Part (a) — Single Branch at I7

Predictor state: **WT** ⇒ predicts **Taken**. Actual outcome: **Not Taken**.

Misprediction. I8 and I9 are flushed. State transitions WT → WNT.

Part (b) — FSM Transition



Part (c) — Five-Iteration Trace: T, T, NT, T, NT

Iteration 1 — Actual: T, State before: WT

- Prediction: Taken Actual: Taken ⇒ **Correct**
- WT → ST

Iteration 2 — Actual: T, State before: ST

- Prediction: Taken Actual: Taken ⇒ **Correct**
- ST → ST (saturated)

Iteration 3 — Actual: NT, State before: ST

- Prediction: Taken Actual: Not Taken ⇒ **Misprediction**
- ST → WT

Iteration 4 — Actual: T, State before: WT

- Prediction: Taken Actual: Taken ⇒ **Correct**
- WT → ST

Iteration 5 — Actual: NT, State before: ST

- Prediction: Taken Actual: Not Taken ⇒ **Misprediction**
- ST → WT

Summary table:

Iter	Actual	State Before	Prediction	State After	Correct?
1	T	WT	Taken	ST	**Yes**
2	T	ST	Taken	ST	**Yes**
3	NT	ST	Taken	WT	**No**
4	T	WT	Taken	ST	**Yes**
5	NT	ST	Taken	WT	**No**

Total mispredictions: 2 (iterations 3 and 5).

The pattern T, T, NT, T, NT keeps the counter oscillating between ST and WT, never reaching WNT. It always predicts Taken and is wrong on every NT outcome. The 2-bit counter has no history beyond its current state and cannot learn the alternating pattern.

Part (d) — Time Lost per Misprediction

$$\begin{aligned} \text{Clock frequency} &= 2 \text{ GHz} \\ \text{Clock period} &= \tfrac{1}{2\times 10^9} = 0.5 \text{ ns} \\ \text{Flush penalty} &= 4 \text{ cycles} \\ \text{Time lost} &= 4 \times 0.5 = \mathbf{2} \text{ ns per misprediction} \end{aligned}$$

With 2 mispredictions: total wasted time $= 2 \times 2 = \mathbf{4}$ ns $= \mathbf{8}$ clock cycles of lost compute time.

4 Q4 — Out-of-Order Scheduling

Setup and Execution Latencies

All instructions enter the reservation station at cycle 3. An instruction issues once all source operands are ready.

Unit	Latency	Result ready
Load/Store (LSU)	2 cycles	issue +2
ALU	1 cycle	issue +1
MUL	3 cycles	issue +3

Structural Hazard on the LSU

Correction note. I1 and I2 are both loads. There is only **one LSU port**. Even though both have no data dependencies and are ready at cycle 3, they *cannot* issue simultaneously. This is a structural hazard. Renaming does not help — the bottleneck is the physical hardware port.

I1 issues cycle 3 $\Rightarrow$ result `p6` ready cycle 5.
I2 issues cycle 4 $\Rightarrow$ result `p7` ready cycle 6.

Instruction-by-Instruction Derivation

Issue cycle = max(all operands ready, execution unit free).

I3: `add p8 ← p6, p7`

- p6 ready cycle 5, p7 ready cycle 6
- $\max(5, 6) = 6$; ALU free at cycle 6

$$\text{Issue} = 6 \qquad \text{Result ready} = 6 + 1 = \mathbf{7}$$

I4: `mul p9 ← p8, p4`

- p8 ready cycle 7, p4 ready cycle 3
- $\max(7, 3) = 7$; MUL free at cycle 7

$$\text{Issue} = 7 \qquad \text{Result ready} = 7 + 3 = \mathbf{10}$$

Key bottleneck. The 3-cycle MUL occupies cycles 7–9. During cycles 8 and 9 the ALU is completely free but no ALU-ready instruction exists. I5 cannot issue until p9 arrives at cycle 10.

I5: `add p10 ← p9, p7`

- p9 ready cycle 10, p7 ready cycle 6
- $\max(10, 6) = 10$; ALU free at cycle 10

$$\text{Issue} = 10 \qquad \text{Result ready} = 10 + 1 = \mathbf{11}$$

I6: `add p11 ← p10, p6`

- p10 ready cycle 11, p6 ready cycle 5
- $\max(11, 5) = 11$; ALU free at cycle 11

$$\text{Issue} = 11 \qquad \text{Result ready} = 11 + 1 = \mathbf{12}$$

Part (a) — Completed Issue Table

Instr	Operation	Operands Ready	Issue Cycle	Result Ready
I1	`lw p6`	cycle 3	3	5
I2	`lw p7`	cycle 3 (LSU stall*)	**4**	**6**
I3	`add p8←p6,p7`	p6@5, p7@6	6	7
I4	`mul p9←p8,p4`	p8@7, p4@3	7	10
I5	`add p10←p9,p7`	p9@10, p7@6	10	11
I6	`add p11←p10,p6`	p10@11, p6@5	11	12

*I2 delayed one cycle: single LSU port already occupied by I1 at cycle 3.

Part (b) — I2 and I5 Both Read `p7`

No conflict. Both instructions *read* from `p7`; reads are non-destructive. The physical register file supports multiple simultaneous reads from the same register. A conflict would only arise if two instructions attempted to *write* the same physical register, which renaming prevents by assigning each destination a unique PRF slot. Since `p7` is never written again after I2, it remains valid indefinitely.

Part (c) — Longest Wait and Critical Path

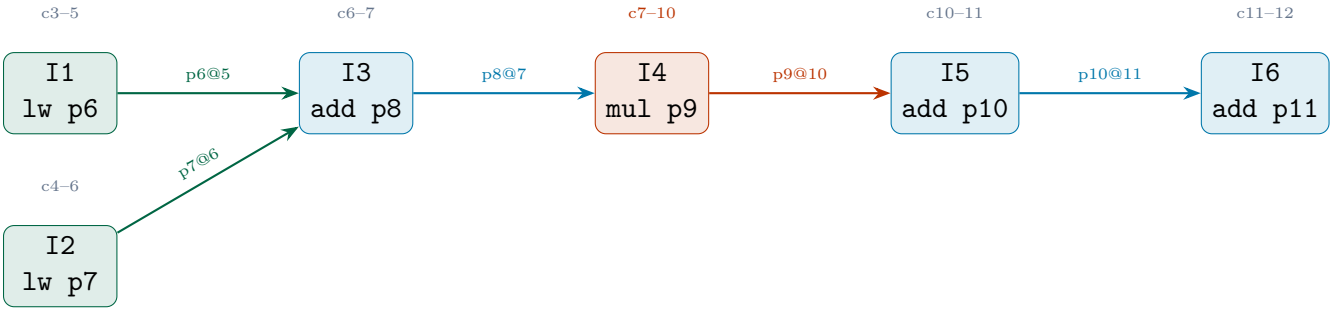
I6 enters the reservation station at cycle 3 but issues at cycle 11 — a wait of **8 cycles**.

Critical path: `I1` → `I3` → `I4` → `I5` → `I6`

Step	Event	Cycle
I1 issues	load begins	3
I1 result ready	`p6` available	5
I3 issues	waits for `p7`@6	6
I3 result ready	`p8` available	7
I4 issues	waits for `p8`@7	7
I4 result ready	`p9` available (3-cyc MUL)	10
I5 issues	waits for `p9`@10	10
I5 result ready	`p10` available	11
I6 issues	waits for `p10`@11	11
I6 result ready	`p11` available	12

Critical path length $= 12 - 3 = \mathbf{9}$ cycles

The MUL at I4 contributes 3 of those 9 cycles and is the single largest source of latency.



5 Q5 — Superscalar Throughput

Part (a) — Valid Co-issue Pairs

For two instructions to co-issue: (1) both ready in that cycle, (2) different execution units, (3) no RAW between them.

Pair	Units	Valid?	Reason
I1 + I2	LSU + LSU	**No**	Single-port LSU — structural hazard. Serialised: I1@c3, I2@c4.
I1 + I3	LSU + ALU	**No**	RAW: I3 needs p6 from I1, not ready until cycle 5.
I1 + I4	LSU + MUL	**No**	I4 depends on I3 which depends on I1. Not ready until c7.
I1 + I5	LSU + ALU	**No**	I5 is four steps down the RAW chain. Not ready until c10.
I1 + I6	LSU + ALU	**No**	I6 reads p6 from I1 directly (RAW) and also needs p10@11.
I2 + I3	LSU + ALU	**No**	RAW: I3 needs p7 from I2.
I3 + I4	ALU + MUL	**No**	RAW: I4 needs p8 from I3. I3@c6, I4@c7 — consecutive only.
I4 + I5	MUL + ALU	**No**	RAW: I5 needs p9 from I4.
I5 + I6	ALU + ALU	**No**	Single ALU (structural) and RAW: I6 needs p10 from I5.

Valid co-issue pairs: none. The 2-wide issue port is entirely unused. Every instruction issues alone.

Part (b) — OoO Stall Savings vs. In-Order

Correction note. The issue schedule below reflects the corrected I2 issue cycle of 4 (not 3) due to the single-port LSU structural hazard. This correction applies equally to both the in-order and OoO columns.

	In-Order Superscalar	OoO Superscalar
I1 issues	cycle 3	cycle 3
I2 issues	cycle 4	cycle 4
I3 issues	cycle 6	cycle 6
I4 issues	cycle 7	cycle 7
I5 issues	cycle 10	cycle 10
I6 issues	cycle 11	cycle 11
Last result ready	cycle 12	cycle 12
Stall cycles	8	8

OoO stall savings: 0 cycles.

The two schedules are identical. Every instruction is gated by the data dependency of its immediate predecessor, not by the in-order constraint. There is no instruction that OoO can pull forward, because every instruction depends directly on the one before it. OoO only provides benefit when independent instructions exist to schedule around a stall — this sequence offers none.

Part (c) — IPC Under OoO Superscalar

$$\begin{aligned} \text{Instructions completed} &= 6 \\ \text{First issue cycle} &= 3 \\ \text{Last result cycle} &= 12 \\ \text{Total cycles elapsed} &= 12 - 3 + 1 = 10 \end{aligned}$$

$$\boxed{\text{IPC} = \frac{6}{10} = 0.60}$$

Cycle-by-cycle slot utilisation:

Cycle	Slot 1	Slot 2
3	I1 (LSU)	*idle*
4	I2 (LSU)	*idle*
5	*idle*	*idle*
6	I3 (ALU)	*idle*
7	I4 (MUL)	*idle*
8	*MUL bubble*	*idle*
9	*MUL bubble*	*idle*
10	I5 (ALU)	*idle*
11	I6 (ALU)	*idle*

Out of $10 \times 2 = 20$ available issue slots, only 6 are used — a slot utilisation of **30%**.

Summary — Three Compounding Limits

1. Superscalar width is bounded by independence. A dependency chain with no independent side work achieves IPC ≤ 1 regardless of issue width. Adding more ports does not help if there is only one ready instruction per cycle.

2. OoO is bounded by the dependency graph. If every instruction waits for the immediately preceding one, the OoO engine has no scheduling freedom and matches in-order performance.

3. Multi-cycle units create bubbles below IPC = 1. The 3-cycle MUL inserts two fully idle cycles out of ten, reducing IPC from a structural ceiling of 1.0 down to 0.60.

To improve IPC the dependency chain must be broken — via instruction interleaving from elsewhere in the function, or software pipelining across loop iterations to overlap a MUL from iteration k with ALU work from iteration $k + 1$.